```
    })
}
```

We've changed the argument of `toggleAssociatedConcerts` to explicitly allow an undefined value, then we return out if the value is undefined, which might be technically unnecessary but does make it explicit that we're dealing with the undefined value.

Finally, the dispatch issue is related to the type of the `VenueThunk` and is fixed by changing the `null` argument in that type definition to `void`. I think that maybe it should have been `void` all along, but certainly with the null checks turned on, TypeScript wants this to be a little bit more explicit.

And now our code compiles again, just a little bit stricter. We've fixed one potential bug in the `CalendarController` and, in the other cases, mostly just made the code more consistent.

These potential errors are minor, but they allow us to change the way we have been thinking about type checking. So far, we've been looking at type checking as a form of communication between the coders and the compiler, telling the compiler about the types so that the compiler can warn you about mismatches between different parts of the code.

Now we can start looking at type checking as a way to prevent run-time errors by making it impossible for invalid states to make it past the compiler. In this case, the invalid state is the case where the selector that we are assuming is in the DOM isn't there. In other words, the compiler here is alerting us to a case that our code doesn't handle, and we need to figure out what we want to do.

Our response to this compiler error depends on how confident we are that the `null` condition won't happen. If we're pretty certain we know more about the code than the compiler does, we can make the compiler error go away by using a type assertion.